6-SQL_for_Analysts

Day 1: Intro to SQL

What is SQL?

SQL = Structured query language

SQL is a programming language used for storing and processing information in a relational database

What is a relational database?

A relational database contains at least two tables.

The tables have rows and columns, like a table in excel.

Each table has 1 or more primary keys, which are columns of values that can be used to identify specific rows in the dataset.

The tables are related to each other, and can be joined based on the primary keys.

Relational Database Example

Imagine a store has two tables: Sales and Products.

Sales contains the information about sales, including time, date, payment method, and product_id

Products contains information about the products, including price, weight, description, and product_id

Sales and Products tables can be joined on the product_id key.

Why Use A Relational Database?

Imagine the example of the store with sales and products.

Imagine they only had one table, where they kept track of sales and information about the product sold.

What would be the problem with this?

Why Use A Relational Database?

This means that for every single sale made, they have to add a row to the table not only of the sale information, but also including all the product details, since they do not have any other table with the product details saved.

Imagine the store has a total of 10 products, but makes 1000s of sales daily.

This would be repetitive and would also waste a lot of storage space.

Let's practice as we learn

We are going to practice queries online.

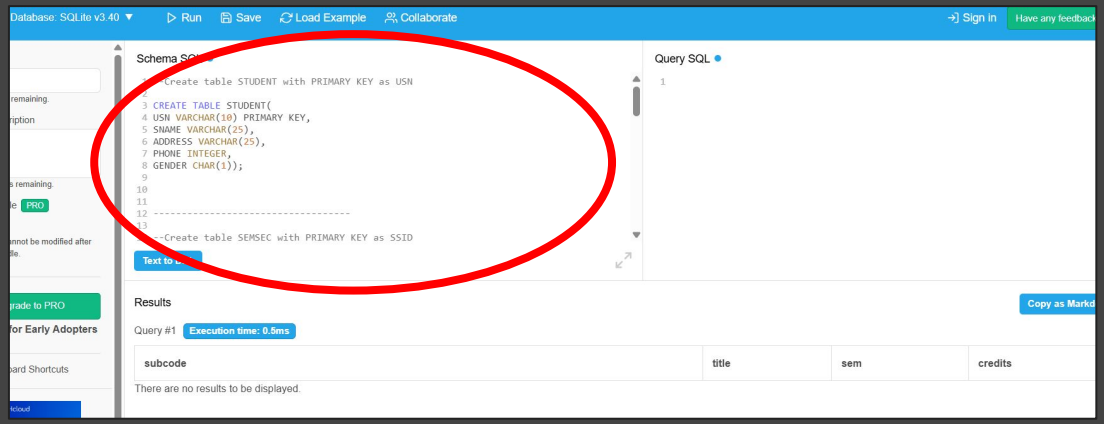
First, go to this website: https://www.db-fiddle.com/

Let's practice as we learn

Let's add the schema in order to run queries.

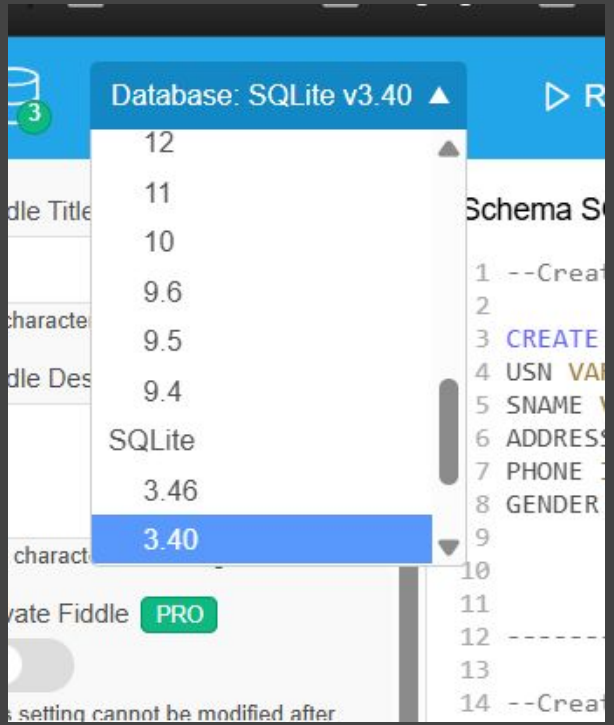
Go to these two links and copy and paste the code into the left side of the db-fiddle page. Do not run it yet.
https://github.com/teamkaliyona/vtu-dbms-lab/blob/main/DBMS-04-College%20Database/College-Create%20Table%20Scripts.sql

https://github.com/teamkaliyona/vtu-dbms-lab/blob/main/DBMS-04-College%20Database/College-Insert%20Scripts.sql



Let's practice as we learn

Set the database in the left corner to sqlite



Let's practice as we learn

Remove all lines that begin with DESC (ex: DESC STUDENT;)

Let's practice as we learn

Now, click run. You will find it on the top left of the page

Let's practice as we learn

Important: Do not delete the code on the left side of db-fiddle.

The tables are not actually saved on the website, so if you delete the create table and insert statements, there will be no tables to query.

SELECT

SELECT & FROM are the building blocks of all SQL queries.

SELECT tells SQL which columns you want to view.

FROM tells SQL which table you want to work with.

Does anyone know what a basic SELECT + FROM statement looks like?

SELECT

Imagine we want to view the columns sname and address from the student table.

How would we do that?

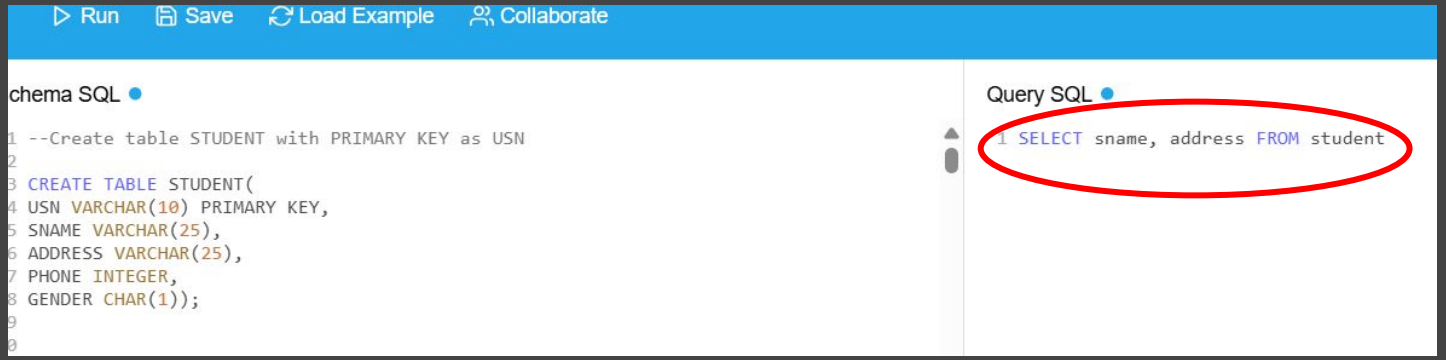
SELECT

View the columns sname and address from the student table:

SELECT sname, address FROM student

Type this statement into the right side of db-fiddle.

Click run in the top left of the page to run the statement and check your work.



SELECT

What if we want to view all the columns from the student table?

How would we do that?

SELECT

What if we want to view all the columns from the student table?
How would we do that?

```
SELECT * FROM student
```

SELECT

What if we want to view the column sname, but we want to call it first_name. How would we do that?

SELECT

What if we want to view the column sname, but we want to call it first_name. How would we do that?

SELECT sname as first_name FROM student

LIMIT

LIMIT is very important if you have a large dataset.

LIMIT sets the maximum number of rows that will be displayed from your query.

Always put LIMIT on the last line of the query

LIMIT

Example - display 2 rows from the student table:

```
SELECT *

FROM student

LIMIT 2
```

WHERE

After SELECT and FROM, WHERE is one of the most common SQL commands.

What is the purpose of WHERE?

WHERE

WHERE is a filter.

Imagine you want to view all columns from the subject table, but only view the rows which have 3 credits. How would you do that?

WHERE

```
SELECT *
FROM subject
WHERE credits =3
```

WHERE

Now imagine you want to view the title and sem (rename the column as semester) from the subject table, but only view the rows which have semester of 4 or more.

How would you do that?

WHERE

```
SELECT title, sem AS semester

FROM subject

WHERE sem >= 4
```

ORDER BY

ORDER BY tells SQL the order you want to use to view rows.

ORDER BY can take a column, or a list of columns.

ORDER BY

Imagine you want to see the student table, with the names ordered alphabetically.

How would you do that?

ORDER BY

Imagine you want to see the student table, with the names ordered alphabetically. How would you do that?

```
SELECT *
FROM student
ORDER BY sname
```

ORDER BY

Now, you want to see the names alphabetically, but from z to a.

What do you need to add to the query?

ORDER BY

Now, you want to see the names alphabetically, but from z to a.

What do you need to add to the query? DESC

```
SELECT *

FROM student

ORDER BY sname DESC
```

ORDER BY

Now, you want to see the iamarks table ordered by both test1 and test2, with test1 the priority order.

How would you do that?

ORDER BY

Now, you want to see the iamarks table ordered by both test1 and test2, with test1 the priority order.

How would you do that?

```
SELECT *

FROM iamarks

ORDER BY test1, test2
```

ORDER BY

Order by can take the column name, or it can take the number in the order in the select statement.

So, these two statements are the same:

Select sname, address from student order by sname

Select sname, address from student order by 1

If you wanted to order by address using the number notation, what would the statement be?

ORDER BY

If you wanted to order by address using the number notation, what would the statement be?

Select sname, address from student order by 2

COUNT, MIN, MAX, AVG

These commands are placed within the select statement, in order to count values, or to find the minimum, maximum, or average value.

How would we find the count of total rows in the student table?

COUNT, MIN, MAX, AVG

How would we find the count of total rows in the student table?

```
SELECT COUNT(*) AS total_rows

FROM student.
```

COUNT, MIN, MAX, AVG

How would we find the count of unique addresses in the student table?

COUNT, MIN, MAX, AVG

How would we find the count of unique addresses in the student table? Using **DISTINCT**

```
SELECT COUNT (DISTINCT address)
FROM student
```

COUNT, MIN, MAX, AVG

How would we find the maximum semester in the subject table?

```
SELECT MAX(sem) AS top_semester
FROM subject
```

COUNT, MIN, MAX, AVG

What about the minimum or average semester?

COUNT, MIN, MAX, AVG

What about the minimum or average semester?

```
SELECT MIN(sem) AS top_semester
FROM subject
```

```
SELECT AVG(sem) AS top_semester
FROM subject
```

Class 1 Review

Choose relevant topics from this list of quizzes:

https://www.w3schools.com/sql/sql_exercises.asp

Practice

https://sqlbolt.com/lesson/filtering_sorting_query_results

https://sqlzoo.net/

Day 1 Homework

Complete 3 SQL SELECT exercises from Mode tutorial

Day 2
GROUP BY, aggregations, and calculations

Day 2

e. RESOURCES:

i. SQL Aggregation Functions Reference

ii. Practice SQL Aggregation Queries

Review Homework

Share and run some of our queries.

GROUP BY

GROUP BY groups one or more sets of data and calculates the aggregate based on the group or groups defined in the GROUP BY statement.

This will make more sense with an example.

GROUP BY

Imagine you want to view the total count of students names for each address. How would you do it?

GROUP BY

Imagine you want to view the total count of students names for each address. How would you do it?

```
SELECT address, COUNT(sname) from student
GROUP BY address
```

GROUP BY

With GROUP BY, you can find the count, max, min, and other aggregations.

Make sure you aggregate all the data which is not being used by the group by. If not, you will usually get an error. (but in db-fiddle, it does not cause an error).

How would you fix the query below?

```
SELECT gender, COUNT(DISTINCT sname), address

FROM student

GROUP BY gender
```

GROUP BY

How would you fix the query below?

- Remove ,address

```
SELECT gender, COUNT(DISTINCT sname)
FROM student
GROUP BY gender
```

GROUP BY

You can group by more than one value, and can show aggregations of more than one value too.

Example:

```
SELECT MAX(phone), MIN(phone), SUM(phone), COUNT(DISTINCT sname), address, gender

FROM student

GROUP BY address, gender
```

Aggregations

What are the aggregations you can do after a group by?

Aggregations

What are the aggregations you can do after a group by?

Count, sum, avg, min, max, count distinct

Note: You can also run these aggregations on a single column, like this:

```
SELECT MAX(date)

FROM sales_data_table
```

Sales and Customers Datasets

Take a few minutes and explore them using SQL and find some insights.

Share what you found with your classmates.

Homework

Write 2 GROUP BY queries on mock dataset

Class 3

b. Theory (Short summary)

i. JOINs: INNER, LEFT, RIGHT

c. Main things (Objectives)

i. Combine tables using SQL joins

ii. Write JOIN queries across 2–3 tables

d. For next day (Homework)

i. Join tables in a mock eCommerce database

e. RESOURCES:

i. SQL JOIN Cheatsheet

ii. JOIN Practice Exercises

Class 3

JOINS

Review Homework

Review the queries we made for homework.

Joins

What type of joins exist in SQL?

Joins

What type of joins exist in SQL?

Inner, left, outer, and right

Note: right joins are almost never used

The purpose of joins

What is the purpose of joins?

The purpose of joins

What is the purpose of joins?

Join two or more tables in order to discover insights about the data

Joins

Think about joining two tables. What do you need in order to join tables?

Think about this example:

If you have a table of customer names, ids, and addresses, could you join it with a table of recipe titles, main ingredients, and cooking times?

Joins

If you want to join two tables, what do they both need?

Joins

In order to join tables, there must be a column with shared values in both tables.

A column with an identifying value is called a primary key. A primary key could be, for example, a unique id.

A foreign key is a primary key placed in another table, for example, product_id is the primary key in the product table, and the foreign key in the sales table.

Often, tables are joined on primary and foreign keys, although they can be joined on other columns too.

Inner Join

What is an inner join?

Inner Join

What is an inner join?

Join only the values that are in both tables.

Inner Join

Which countryIDs (ID in the right table) will be in the inner join merged table?

Left Table

Date	CountryID	Units
1/1/2024	1	40
1/2/2024	1	25
1/3/2024	3	30
1/4/2024	2	35

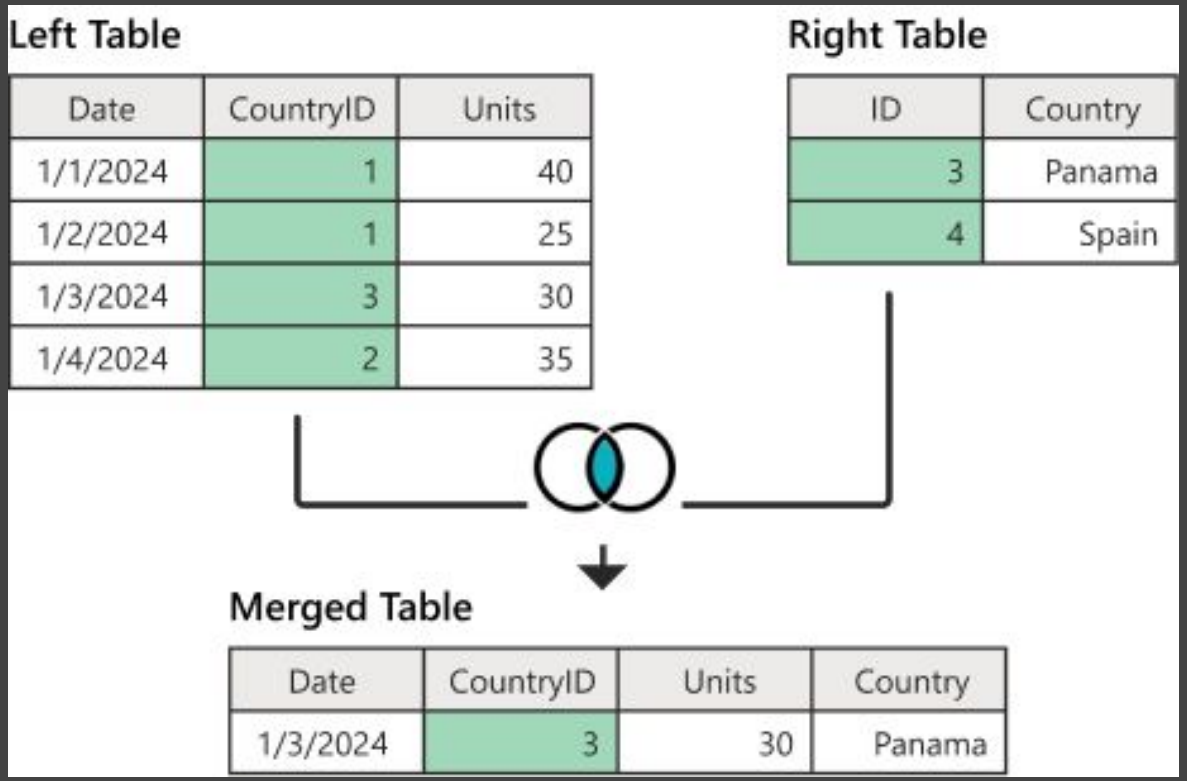
Right Table

ID	Country
3	Panama
4	Spain

Merged Table

Inner Join

If you join two tables using an inner join, the resulting table will have only rows in which the join key value is in both tables.



Left Join

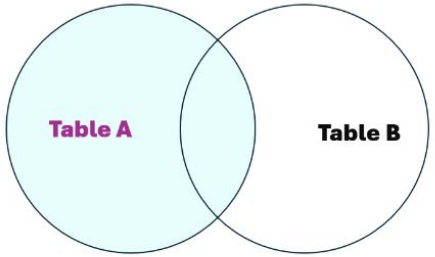
What is a left join?

Left Join

Which IDs will be in the resulting table after the left join?

Table A

ID	Product
1	Chair
2	Table
3	Laptop
4	Phone
5	Flash Drive




```
SELECT  *
FROM  Table A as a
LEFT JOIN  Table B as b
ON a.ID = b.ID
```

Table B

ID	Price
1	$80
2	$120
4	$799
6	$99

Left Join

All values on the table written first (the left) are preserved.

Table A

ID	Product
1	Chair
2	Table
3	Laptop
4	Phone
5	Flash Drive

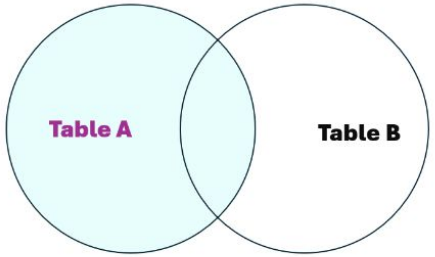


Table B

ID	Price
1	$80
2	$120
4	$799
6	$99

```
SELECT  *
FROM  Table A as a
LEFT JOIN Table B as b
ON a.ID = b.ID
```

ID	Product	ID	Price
1	Chair	1	$80
2	Table	2	$120
3	Laptop	NULL	NULL
4	Phone	4	$799
5	Flash Drive	NULL	NULL

Outer Join

What is an outer join?

Outer Join

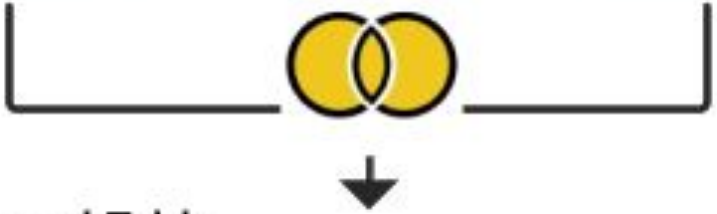
What will the merged table look like?

Left Table

Date	CountryID	Units
1/1/2024	1	40
1/2/2024	1	25
1/3/2024	3	30
1/4/2024	2	35

Right Table

ID	Country
1	USA
2	Canada
3	Panama
4	Spain



Merged Table

Outer Join

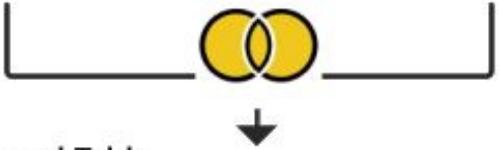
In an outer join, all rows are preserved.

Left Table

Date	CountryID	Units
1/1/2024	1	40
1/2/2024	1	25
1/3/2024	3	30
1/4/2024	2	35

Right Table

ID	Country
1	USA
2	Canada
3	Panama
4	Spain



Merged Table

Date	CountryID	Units	Country
1/1/2024	1	40	USA
1/2/2024	1	25	USA
1/4/2024	2	35	Canada
1/3/2024	3	30	Panama
null	*null*	*null*	Spain

Right Join

What is a right join?

Right Join

All values on the second table in the join statement (the table written on the right) are preserved.

Table A

ID	Product
1	Chair
2	Table
3	Laptop
4	Phone
5	Flash Drive

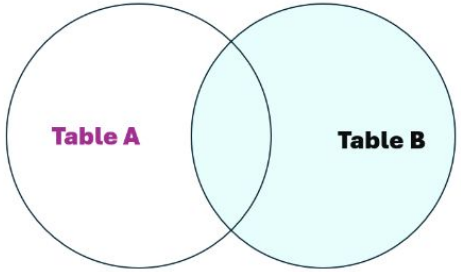


Table B

ID	Price
1	$80
2	$120
4	$799
6	$99

```
SELECT  *
FROM  Table A as a
RIGHT JOIN  Table B as b
ON a.ID = b.ID
```

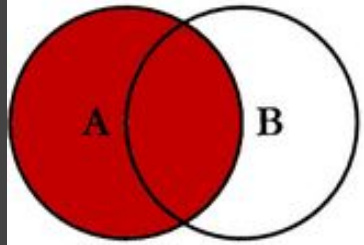
ID	Price	ID	Product
1	$80	1	Chair
2	$120	2	Table
4	$799	4	Phone
6	$99	NULL	NULL

Right Join

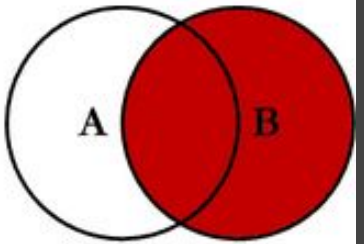
Right joins are almost never used by programmers. They prefer left joins.

A right join and a left join can do the same thing - all you have to do is switch the order you write the tables in to change a right join to a left join.

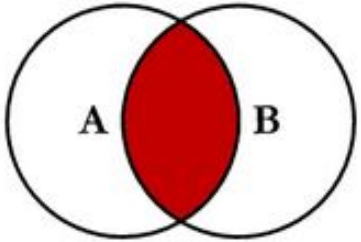
SQL JOINS



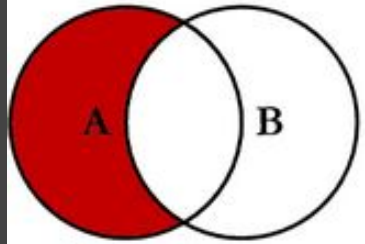

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



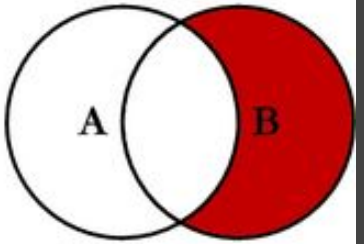

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



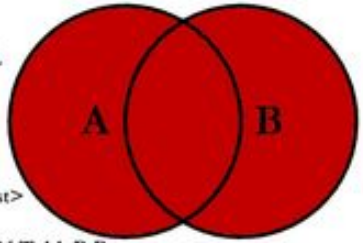

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



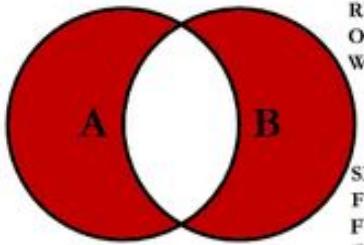

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

© C.L. Moffatt, 2008

Joins Resources

For more information about joins, read this article, which discusses other more complex types of joins and how to do them in SQL

https://www.thedataschool.co.uk/le-luu/sql-joins/

Joins questions

Let's use this website to practice joins

https://www.codechef.com/learn/course/sql-intermediate/SQOOBS01/problems/ASQL01D

The website has exercises, but first let's do some queries together. (The website will say the answer is wrong, but that is okay, we can still see the results)

Joins Queries

We want to see the ids and names of only the customers who made orders.

How would we write the query?

Joins Queries

We want to see the ids and names of only the customers who made orders. How would we write the query?

```
SELECT customers.customer_id, customer_name
FROM customers
INNER JOIN orders
ON orders.customer_id=customers.customer_id
```

Notice we have to write the table name before the column name if the column name appears in both tables

Join Queries

```
SELECT cust.customer_id, customer_name
FROM customers cust
INNER JOIN orders o
ON o.customer_id=cust.customer_id
```

We can also substitute the table name with a letter or abbreviation.

Join Queries

How would you write the query to find all product_names, and the total quantity of each product ordered?

Join Queries

Find all product_names, and the total quantity of each product ordered:

```
SELECT products.product_name,
SUM(orders.quantity) AS total_orders
FROM products
LEFT JOIN orders
ON orders.product_name=products.product_name
GROUP BY 1
```

Notice it says GROUP BY 1.
You could also say GROUP BY products.product_name.

Joins Queries

We want to see all products, and all orders.

Joins Queries

We want to see all products, and all orders.

```
Select * from products
Full outer join orders
on orders.product_name=products.product_name
```

Joins Queries

Now, let's practice doing the exercises on the website.

Homework: Joins

Create a business scenario where your boss asks you a question related to data, and you need to use joins to answer it.

Create two scenarios, and create the answers for them.

You can use the free online dataset we looked at in class.

Class 4

Review Homework

Review Joins Homework

Subqueries

What is a subquery?

Subqueries

A subquery is a helper query written within a main query

Subqueries

Let's use the data on this website again: https://www.codechef.com/learn/course/sql-intermediate/SQOOBS01/problems/ASQL01D

What is the query below finding?

```
SELECT *

FROM products

WHERE price = (SELECT MAX(price) FROM products)
```

Subqueries

What is the query below finding?

```
SELECT *
FROM products
WHERE price = (SELECT MAX(price) FROM products)
```

-> All product information about the highest priced products

Subqueries

What is the query below finding?

```
SELECT product_name

FROM products

WHERE product_name IN (SELECT product_name FROM orders
WHERE quantity = 1)
```

Subqueries

What is the query below finding?

```
SELECT product_name

FROM products

WHERE product_name IN (SELECT product_name FROM orders
WHERE quantity = 1)
```

-> It is finding the names of the products which had an order quantity of 1.

Subqueries

Now try one yourself - Can you find the price of the item ordered most recently?

Subqueries

Now try one yourself - Can you find the price of the item ordered most recently?

```
SELECT price

FROM products

WHERE product_name IN (SELECT product_name FROM orders
WHERE order_date = (SELECT MAX(order_date) FROM orders))
```

Subqueries, Nested Queries

The subquery we looked at on the last slide is an example of a nested query:

```
SELECT price
FROM products
WHERE product_name IN (SELECT product_name
                       FROM orders
                       WHERE order_date =
                       (SELECT MAX(order_date) FROM orders)
)
```

Common Table Expression (CTE)

Link multiple table logics together in order to get what you want

Common Table Expression (CTE): Example

```
WITH customer_orders AS (
    SELECT
        customer_id,
        SUM(quantity) AS total_quantity
    FROM orders
    GROUP BY customer_id
)
SELECT
    c.customer_id,
    c.customer_name,
    customer_orders.total_quantity
FROM customers c
JOIN customer_orders
    ON c.customer_id = customer_orders.customer_id
ORDER BY total_quantity DESC;
```

CTE

Write a SQL query using a CTE to find the total amount spent by each customer on Electronics products only.

Display:

customer_id , customer_name, total_spent

Order the results from highest to lowest total spent.

CTE Solution

```sql
WITH electronics_spending AS (
    SELECT
        o.customer_id,
        SUM(o.quantity * p.price) AS total_spent
    FROM orders o
    JOIN products p
        ON o.product_name = p.product_name
    JOIN categories c
        ON p.category_id = c.category_id
    WHERE c.category_name = 'Electronics'
    GROUP BY o.customer_id
)

SELECT
    cu.customer_id,
    cu.customer_name,
    electronics_spending.total_spent
FROM customers cu
JOIN electronics_spending
    ON cu.customer_id = electronics_spending.customer_id
ORDER BY total_spent DESC;
```

CTE

You can stack many tables, like this:

```
WITH table1 AS (
    SELECT
        .....
),
table2 AS (
    SELECT
        .....
),
table3 AS (
    SELECT
        .....
)
SELECT * FROM table3
```

Conditional logic in SQL: CASE WHEN

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN conditionN THEN resultN
    ELSE result
END;
```

CASE WHEN: Example

```
SELECT first_name, department_id, salary,
    CASE department_id
        WHEN 50 THEN 1.5 * salary
        WHEN 12 THEN 2.0 * salary
        ELSE salary
    END AS "REVISED SALARY"
FROM Employee;
```

CASE WHEN Exercise

Write a SQL query that shows customer_name, total_spent, and a spending category based on total spent, according to the table in the next slide

CASE WHEN exercise

Condition	Category
total_spent ≥ 2000	'VIP'
total_spent ≥ 1000	'Regular'
total_spent < 1000	'Occasional'
No orders	'No Orders'

CASE WHEN Solution

```
SELECT
    c.customer_name,
    SUM(o.quantity * p.price) AS total_spent,
    CASE
        WHEN SUM(o.quantity * p.price) >= 2000 THEN 'VIP'
        WHEN SUM(o.quantity * p.price) >= 1000 THEN 'Regular'
        WHEN SUM(o.quantity * p.price) > 0 THEN 'Occasional'
        ELSE 'No Orders'
    END AS spending_category
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
LEFT JOIN products p
    ON o.product_name = p.product_name
GROUP BY c.customer_name;
```

COALESCE

The COALESCE Function in SQL

The COALESCE function returns the first non-null argument. It is particularly useful for handling NULL values and substituting them with a default value.

Syntax

COALESCE(value1, value2, ..., valueN)

COALESCE

Example Query:

```
SELECT COALESCE(last_name, '- NA -')
FROM Employee;
```

What will this return if last_name is Null?

COALESCE

Example Query:

```
SELECT COALESCE(last_name, '- NA -')
FROM Employee;
```

What will this return if last_name is Null?

It will return - NA -

COALESCE exercise

Go back to the CASE WHEN exercise and rewrite it using coalesce

CASE WHEN with COALESCE Solution

```
SELECT
    c.customer_name,
    COALESCE(SUM(o.quantity * p.price), 0) AS total_spent,
    CASE
        WHEN COALESCE(SUM(o.quantity * p.price), 0) >= 2000 THEN 'VIP'
        WHEN COALESCE(SUM(o.quantity * p.price), 0) >= 1000 THEN 'Regular'
        WHEN COALESCE(SUM(o.quantity * p.price), 0) > 0 THEN 'Occasional'
        ELSE 'No Orders'
    END AS spending_category
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
LEFT JOIN products p
    ON o.product_name = p.product_name
GROUP BY c.customer_name
ORDER BY total_spent DESC;
```

Calculated fields

You can create a new column based on an operation done on existing columns

For example:

SELECT field1+field2 AS fields_sum FROM table...

Homework

Create 2 CASE WHEN queries and 1 subquery

Class 5

Window functions: ROW_NUMBER, RANK, OVER()

What is a window function?

Window functions: ROW_NUMBER, RANK, OVER()

An SQL Window Function performs calculations across a set of rows that are related to the current row without "collapsing" them.

Think of it as looking through a "window" at a specific group of rows while still keeping every individual row in your final list.

Window functions versus group by

Similar to GROUP BY, however:

GROUP BY: Fuses many rows into one single summary row. You lose the individual details.

Window Function: Keeps every row as it is. It just adds a new column next to them containing the calculation result

Window function syntax

The Core Syntax: OVER()

The OVER() clause is what tells SQL you are using a window function. It has three optional parts:

PARTITION BY: Splits your data into groups (like "windows" for each department or category).

ORDER BY: Sets the order of rows within those groups (essential for things like running totals or rankings).

Rows/Range (Frame): Defines exactly which rows around the current one to look at (e.g., "the 3 rows before this one").

Window function syntax

```
function_name(expression) OVER (
  [PARTITION BY column]
  [ORDER BY column]
  [ROWS/RANGE frame_specification]
)
```

Types of Window functions

Common Types of Window Functions

Aggregate Functions: Use standard math like SUM(), AVG(), or COUNT() but across a window.

Example: Calculate a "running total" of sales as you go down the list.

Ranking Functions: Assign a number to each row based on its position.

Example: ROW_NUMBER() gives each row a unique number; RANK() can handle ties.

Value (Offset) Functions: Let you "peek" at other rows without using a join.

Example: LAG() looks at the row above (useful for comparing today's price to yesterday's); LEAD() looks at the row below

Window functions: ROW_NUMBER, RANK, OVER()

Useful resource:
https://www.codecademy.com/resources/docs/sql/window-functions

Window functions practice

Go to:
https://www.codechef.com/learn/course/sql-intermediate/SQOOBS01/problems/ASQL01D

Group the orders table by the order date. Show the product, the order date, and the total quantity of orders on that day.

Window functions practice

Group the orders table by the order date. Show the product, the order date, and the total quantity of orders on that day.

```
Select product_name, order_date,

sum(quantity) OVER (

  PARTITION BY order_date

  ORDER BY order_date

)

 from orders
```

ROW_NUMBER

From the products table, group the products by category id, and put the categories in order based on average price. Add a row number.

ROW_NUMBER

From the products table, group the products by category id, and put the categories in order based on average price. Add a row number.

```
SELECT
    category_id,
    AVG(price) AS avg_price,
    ROW_NUMBER() OVER (
        ORDER BY AVG(price)
    ) as category_rank
FROM products
GROUP BY category_id;
```

ROW_NUMBER

From the orders table, group the products by product name, and put the categories in order based on average quantity. Add a row number.

ROW_NUMBER

From the orders table, group the products by product name, and put the categories in order based on average quantity. Add a row number.

```
SELECT

    product_name,

    AVG(quantity) AS avg_quantity,

    ROW_NUMBER() OVER (

        ORDER BY AVG(quantity)

    ) as quantity_rank

FROM orders

GROUP BY product_name;
```

ROW_NUMBER

From the orders table, group the products by product name, and put the categories in order based on average quantity. Add a row number.

Notice that the row number goes from 1 to 4, even though the avg quantity stays the same.

What if we wanted the row number to stay the same if the quantity is not increasing? (So if two products have 3 average orders, both would have the same row number)

RANK

What if we wanted the row number to stay the same if the quantity is not increasing? (So if two products have 3 average orders, both would have the same row number)

Change row_number to rank in the last query and see what happens

Use Multiple window functions

You can put multiple windows functions in the same query.

You want to display the min and max order date for each product from the orders table. How?

Use Multiple window functions

You want to display the min and max order date for each product from the orders table:

```
SELECT distinct
    product_name,
    max(order_date)
    OVER (
        partition by product_name
    ) as max_date,
        min(order_date)
    OVER (
        partition by product_name
    ) as min_date
FROM orders
```

Use SQL to perform calculations across partitions

Add a column, price_diff, which is the max price minus the min price.

Use SQL to perform calculations across partitions

Add a column, price_diff, which is the max price minus the min price.

```
SELECT DISTINCT
  category_id,
  MAX(price) OVER w AS max_price,
  MIN(price) OVER w AS min_price,
  MAX(price) OVER w - MIN(price) OVER w AS price_diff
FROM products
WINDOW w AS (PARTITION BY category_id);
```

Homework

Practice 3 window function problems

Class 6

SQL mini-project

Review Homework

Window function solutions

In-class assignment

Mini-project: Write a SQL report for business KPIs using a mock database (PostgreSQL or SQLite) - You choose the data.

As we work, we will stop and share what we have created so far.